

Memory Safety

Slides adapted from: Luca Arnaboldi and David Aspinall,
Informatics@Edinburgh

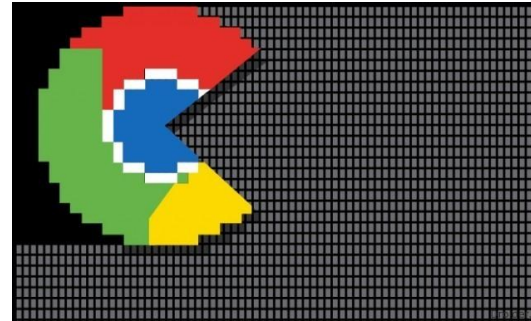
Mihai Ordean

Summary

- Programming and assembly catchup
- Common classes of memory safety issues
- Countermeasures

Memory corruption vulnerabilities

Memory corruption.



- Memory corruption vulnerabilities let the attacker cause the program to write to areas of memory (or write certain values) that the programmer did not intend.
- In the worst cases, these can lead to arbitrary command execution under the attacker's control.

Reasons for memory corruption

Memory corruption vulnerabilities arise from possible:

buffer overflows, in different places

- stack overflows
- heap overflows

other programming mistakes

- pointer arithmetic mistakes
- type confusion errors

Memory safety and undefined behaviour

Memory safety: A programming language enforces memory safety if it ensures that reads and writes stay within clearly defined memory areas.

Undefined behaviour: A programming language specification describes the meaning of programs. For languages that lack memory safety, the spec may say the meaning of an illegal memory access is undefined.

Follow-up questions:

Question. Is there an advantage in "undefined behaviour"?

Question. What risks do you see for software security with "undefined" behaviour?

Programming in C or Assembly

- low-level programs manipulate memory directly
 - Advantage: efficient, precise
 - Disadvantage: easy to violate data abstractions
- arbitrary access to memory
 - pointers and pointer arithmetic
 - mistakes violate memory safety

Memory safety

- memory safety ensures that reads and writes stay within clearly defined memory areas, belonging to different parts of the program.
- Memory areas are often delineated with types and a typing discipline (next week).

Processes and memory

A process is a running program managed by the operating system.

Processes are organized into several memory areas:

- **Code** where the compiled program (or shared libraries) reside.
- **Data** where non-local program variables are stored. This contains global or static variables and the program heap for dynamically allocated data.
- **Stack** which records dynamically allocated data for each of the currently executing functions/methods. This includes local variables, the current object reference (often) and the return address.

The OS (with the CPU, language runtime) can provide varying amounts of protection between these areas.

Instant C programming

- ▶ Java and C use a similar syntax.
- ▶ C has no objects but
 - pointers to memory locations (&val, *ptr)
 - arbitrary-length strings, terminated with ASCII NUL
 - fixed-size structs for records of values
 - explicit dynamic allocation with malloc()
- ▶ C has no exceptions but
 - function return code conventions
- ▶ C is generally more relaxed
 - about type errors
 - uninitialized variables
- ▶ Modern C compilers give strong warnings
 - these days, errors by default
 - can instrument C code with debug/defence code

Instant C programming

```
#include <stdio.h>

void main(int argc, char *argv[]) {
    int c;

    printf("Number of arguments passed: %d\n", argc);

    for (c = 0; c < argc; c++) {
        printf("Argument %d is: %s\n", c, argv[c]);
    }
}
```

Instant C programming

```
$ gcc showargs.c -o showargs
$ ./showargs this is my test
Number of arguments passed: 5
Argument 0 is: ./showargs
Argument 1 is: this
Argument 2 is: is
Argument 3 is: my
Argument 4 is: test$
```

Instant C programming

Exercise. If you haven't programmed C before, try some simple programs from "The Internet". e.g. Write a program to reverse its list of argument words.

Instant Assembly programming

- ▶ x86: hundreds of instructions! But in families:
 - Data movement: MOV ...
 - Arithmetic: ADD, FDIV, IDIV, MUL, ...
 - Logic: AND, OR, XOR, ...
 - Control: JMP, CALL, LEAVE, RET, ...
- ▶ General registers are split into pieces:
 - 32 bits : EAX (extended A) (*or RAX for 64bits*)
 - 16 bits : AX
 - 8 bits : AH AL (high and low bytes of A)
- ▶ Others are pointers to segments, index offsets
 - ESP: stack pointer
 - EBP: base pointer (aka frame pointer)
 - ESI, EDI: source, destination index register

(We'll stick to x86 32-bit instructions in this module, 64-bit is slightly different).

Instant Assembly programming

Here is a file movc.c:

```
int value;  
int *ptr;  
  
void main() {  
    value = 7;  
    ptr = &value;  
    *ptr = value*13;  
}
```

Compile this to assembly code with:

```
$ gcc -S -masm=intel movc.c
```

This produces a file movc.s

Instant Assembly programming

	AT&T	Intel
Parameter order	<pre>movl \$5, %eax</pre> Source before the destination.	<pre>mov eax, 5</pre> Destination before source.
Parameter size	<pre>addl \$0x24, %esp movslq %ecx, %rax padd %xmm1, %xmm2</pre> Mnemonics are suffixed with a letter indicating the size of the operands: <i>q</i> for qword (64 bits), <i>l</i> for long (dword, 32 bits), <i>w</i> for word (16 bits), and <i>b</i> for byte (8 bits).^[6]	<pre>add esp, 24h movsxd rax, ecx padd xmm2, xmm1</pre> Derived from the name of the register that is used (e.g. <i>rax</i>, <i>eax</i>, <i>ax</i>, <i>al</i> imply <i>q</i>, <i>l</i>, <i>w</i>, <i>b</i>, respectively). Width-based names may still appear in instructions when they define a different operation. • MOVSSXD refers to sign extension with dword input, unlike MOVSSX. • SIMD registers have width-named instructions that determine how to split up the register. AT&T tends to keep the names unchanged, so PADD is not renamed to "paddl".

Instant Assembly programming

```
.data
    value    DD 2
    ptr      DD 2

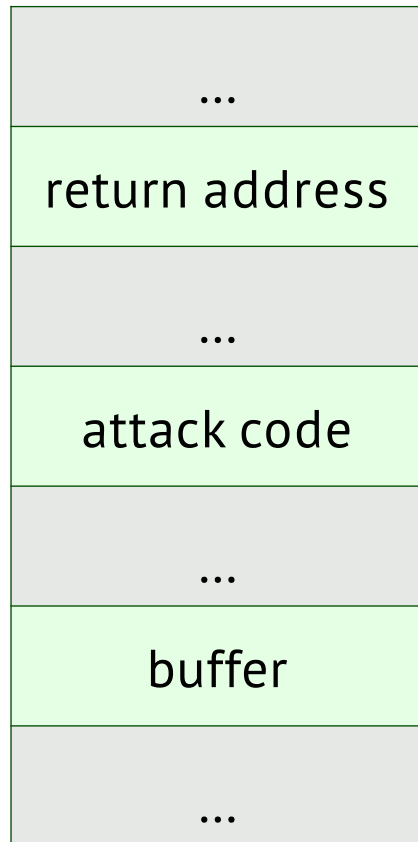
.code
    mov      eax, 7          ; set EAX to 7
    mov      [value], ax     ; value is now 7
    mov      [ptr], value    ; set ptr = address of value
    mov      ecx, [ptr]      ; ECX to same; ECX handles *ptr
    mov      edx, [value]    ; EDX = 7; EDX handles *value
    mov      eax, edx        ; EAX = 7; EAX modifies EDX=*value
    add      eax, eax        ; EAX = 14 (2*7);
    add      eax, edx        ; EAX = 21 (3*7);
    shl      eax, 2          ; EAX = 21 * 4 = 84 (12*7)
    add      eax, edx        ; EAX = 7 + 84 = 91 (13*7)
    mov      [ecx], eax      ; set value = 91; ECX receives the
                             ; finished result from EAX
```

Exercise. If you haven't looked at assembly before, compile some small C programs and try understand the compiled assembler, at least roughly.

Fun and profit

- ▶ **Stack overflow attacks** were first carefully explained in Smashing the stack for fun and profit, a paper written by Aleph One for the hacker's magazine Phrack, issue 49, in 1996.
- ▶ Stack overflows are mainly relevant for C, C++ and other “unsafe” languages with raw memory access (e.g., pointers and pointer arithmetic).
- ▶ Languages with built-in memory safety such as Java, C#, Python, etc., are immune to the worst attacks — providing their language runtimes and native libraries have no exploitable flaws.

Stack overflow: high level view



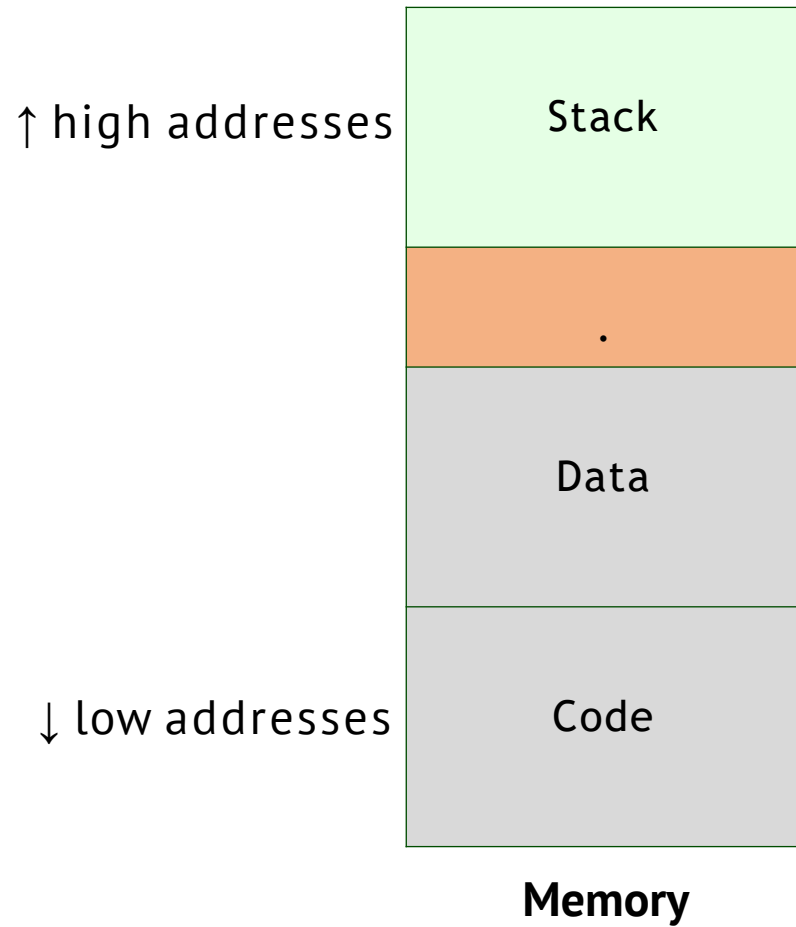
- The malicious argument overwrites all of the space allocated for the buffer, all the way to the return address location.
- The return address is altered to point back into the stack, somewhere before the attack code.
- Typically, the attack code executes a shell.

How the stack works

- ▶ The program stack (i.e., function stack, runtime stack)
 - holds stack frames (i.e., activation records) for each function that is invoked
 - very common mechanism for high-level language implementations
- ▶ So has special CPU support
 - stack pointer registers: on x86, ESP
 - frame pointer registers: on x86, EBP
 - push and pop machine instructions
- ▶ Exact mechanisms vary by CPU, OS, language, compiler, compiler flags.

NB: in pictures, some authors draw stacks growing upwards on the page!
Here we will draw stacks that grow downwards.

How the stack works



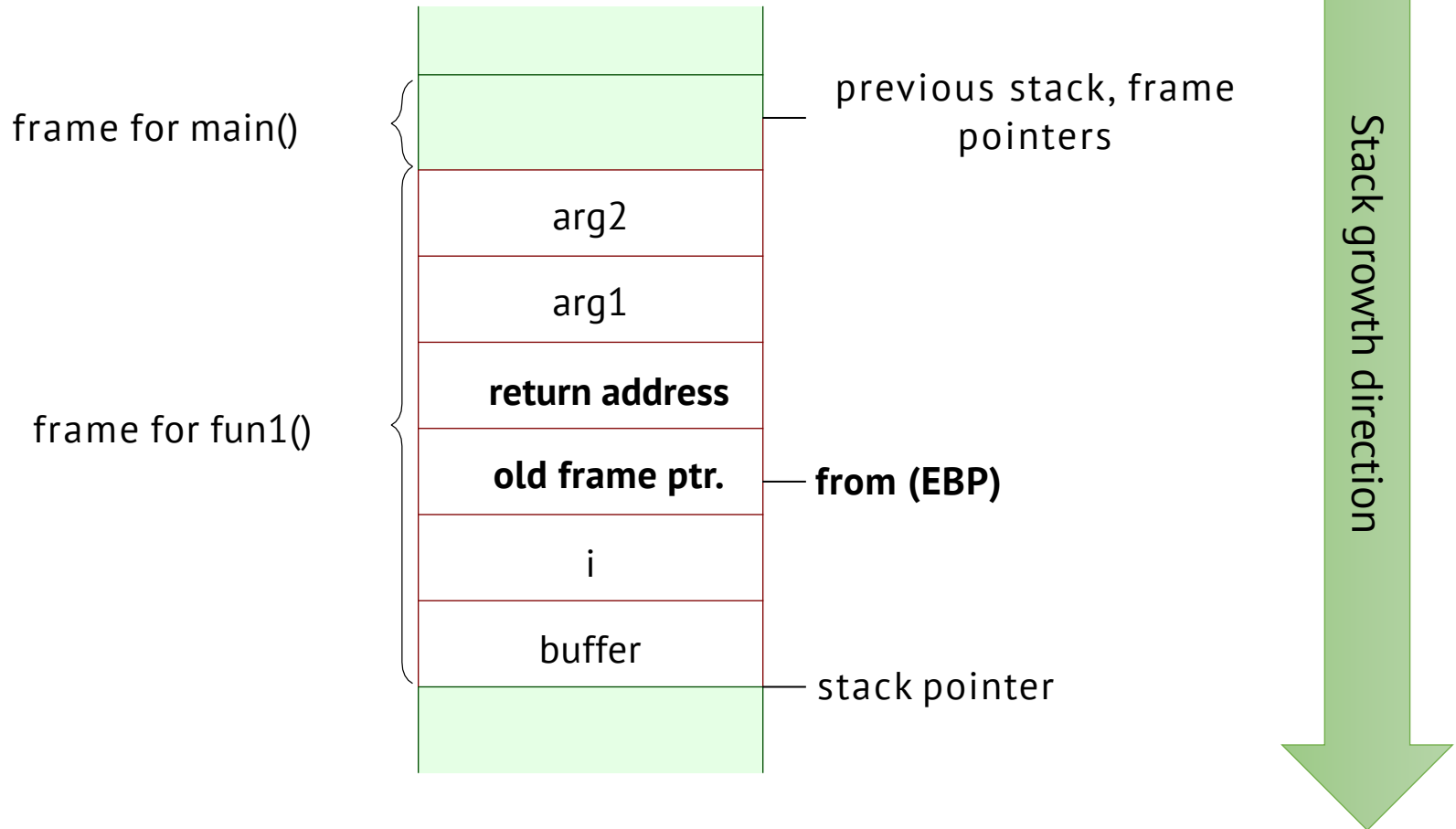
Stack usage with function calls

- ▶ **fun1** has two arguments **arg1** and **arg2**.

```
void fun1 (char arg1, int arg2) {  
    char *buffer[5];  
    int i;  
    ...  
}
```

- ▶ Actual parameters may be passed to the function body on the stack or in registers; the precise mechanism is called **calling convention**.
 - fun1 has two local variables **buffer** and **i**
- ▶ Local variables are allocated space on the stack.
- ▶ A frame pointer may be used to help locate arguments and local variables.

Frame pointers



Assembly code for function calls

- Let's see assembly code produced by gcc compiling C programs on Linux (here 32bit x86), using gcc-S.

```
int main() {  
    return 0;  
}
```

produces

```
main:  
    push ebp                ; save EBP, the old frame pointer  
    mov ebp, esp            ; the new frame pointer for body of main()  
    mov eax, 0              ; the return value in EAX  
    pop ebp                 ; restore the old frame pointer  
    ret
```

Assembly code for function calls

```
void fun1(char arg1, int arg2) {  
    char *buffer[5];  
    int i;  
    *buffer[0] = (char)i;  
}  
  
void main() {  
    fun1('a', 77);  
}
```

Assembly code for function calls

```
fun1:
    push ebp                ; save previous frame pointer
    mov ebp, esp            ; set new frame pointer
    sub esp, 36              ; allocate enough space for locals
    mov eax, [ebp - 24]      ; EAX = address of buffer[0]
    mov edx, [ebp - 4]       ; EDX = i
    mov [eax], dl            ; Set buffer[0] to be low byte of i
    leave                   ; drop frame
    ret                     ; return

main:
    push ebp                ; save previous frame pointer
    mov ebp, esp            ; set new frame pointer
    sub esp, 8               ; allocate space for fun1 parameters
    mov [esp + 4], 77        ; store arg2
    mov [esp], 97            ; store arg1 (ASCII 'a')
    call fun1                ; invoke fun1
    leave                   ; drop frame
    ret                     ; return
```

- **Exercise.** Draw the detailed layout of the stack when the frame for 'fun1()' is active.